

Task 5: .NET Core Web API

Intern: Milan Tiwari | Organization: 366Pi Dev | Date: March 13, 2026 | Status: **COMPLETED**

1. Objective

Develop a **.NET Core Web API** to serve the rig data stored in the local MySQL database (synced in Task 4) to front-end applications via **JSON endpoints**.

MySQL Database (Data) → .NET Web API → JSON Response (Front-end)

2. Technology Stack

Technology	Purpose	Why This Choice
.NET 8 (C#)	API Framework	Minimal APIs for clean, high-performance code.
Dapper ORM	Database Access	Micro-ORM for fast, plain-SQL data retrieval.
MySQL.Data	MySQL Connector	Official driver for MySQL connectivity.
Swagger/OpenAPI	Testing & Documentation	Interactive UI for discovering and testing endpoints.
CORS Policy	Security	Enabled to allow the React SPFx form to call the API.

3. Implementation Details

- **Scaffolded Project:** Created `RigDataApi` using the .NET 8 Web API template.
- **Data Model:** Defined `RigCompany` class matching the MySQL table schema.
- **Endpoints Created:**
 - `GET /api/rigs` — Returns all 9 companies as a JSON array.
 - `GET /api/rigs/{id}` — Returns a single company by its unique ID.
 - `GET /api/rigs/search?name=X` — Search companies by name.
- **Swagger UI:** Configured for automated documentation at `/swagger`.

4. Evidence of Success

Swagger UI Documentation

 **Swagger**
Supported by SMARTBEAR

Select a definition **RigDataApi v1** 

Rig Data API **v1** **OAS 3.0**

<http://localhost:5050/swagger/v1/swagger.json>

RigDataApi

GET **/api/rigs** 

GET **/api/rigs/{id}**  

GET **/api/rigs/search** 

API JSON Response

Pretty print ☐

```
[{"id":1,"companyName":"Noble Corporation
plc","headquartersAddress":"3rd Floor, 1 Ashley Road, Altrincham,
Cheshire, UK (WA14
2DT)","operationalRigs":37,"operationalJackUpRigs":12,"operationalMODUs":25,"officialWebsite":"noblecorp.com"},
{"id":2,"companyName":"Transocean
Ltd.","headquartersAddress":"Transocean Ltd, Turmstrasse 30, CH-6312
Steinhausen,
Switzerland","operationalRigs":27,"operationalJackUpRigs":0,"operationalMODUs":27,"officialWebsite":"deepwater.com"},
{"id":3,"companyName":"Valaris Limited","headquartersAddress":"5847
San Felipe, Suite 3300, Houston, TX 77057,
USA","operationalRigs":51,"operationalJackUpRigs":35,"operationalMODUs":16,"officialWebsite":"valaris.com"}, {"id":4,"companyName":"Shell
plc","headquartersAddress":"Shell Centre, London SE1 7NA, United
Kingdom","operationalRigs":0,"operationalJackUpRigs":0,"operationalMODUs":0,"officialWebsite":"shell.com"},
{"id":5,"companyName":"","headquartersAddress":"","operationalRigs":0,
"operationalJackUpRigs":0,"operationalMODUs":0,"officialWebsite":""},
{"id":6,"companyName":"Vantage Drilling Intl.
Ltd.","headquartersAddress":"Emaar Business Park, Bldg 1, Office 519-
527, The Greens, PO BOX 282292, Dubai,
UAE","operationalRigs":1,"operationalJackUpRigs":0,"operationalMODUs":2,"officialWebsite":"vantagedrilling.com"},
{"id":7,"companyName":"Shelf Drilling Ltd.","headquartersAddress":"One
JLT, Floor 12, Jumeirah Lakes Towers, P.O. Box 212201, Dubai,
UAE","operationalRigs":30,"operationalJackUpRigs":30,"operationalMODUs":0,"officialWebsite":"shelfdrilling.com"},
{"id":8,"companyName":"Milan's Test Rig","headquartersAddress":"123
Ocean
Ave","operationalRigs":0,"operationalJackUpRigs":0,"operationalMODUs":0,"officialWebsite":""}, {"id":9,"companyName":"Milan RIG
TEST","headquartersAddress":"HSR","operationalRigs":1,"operationalJackUpRigs":2,"operationalMODUs":3,"officialWebsite":"366pi.tech"}]
```

5. Challenges & Solutions

Challenge

Solution

DEPENDENCY

Browser CORS Errors		Configured a CORS Policy in Program.cs to allow requests from the SPFx front-end.
VERSIONING Version Conflict	Node.js	Discovered during manual run testing that Task 3 required Node v18 while system was on v24. Installed NVM to manage and swap versions.
ARCHITECTURE Data Retrieval	Fast	Selected Dapper instead of Entity Framework to minimize overhead and ensure lightning-fast JSON responses.

6. Key Learnings

- **Minimal APIs** — Understanding how to build modern, lightweight microservices without MVC complexity.
- **JSON Serialization** — How .NET automatically converts C# objects into JSON for the browser.
- **Interactive Documentation** — Using Swagger to provide a user-friendly interface for API testers.
- **Unified Data Access** — Successfully bridging the gap between local database storage and front-end consumption.